Session 16 · Tuesday 20 October 2026 · Posted after class. Every table here is computed by posb.logic, not typed.

Ticks, glitches and depth — answers

Item 1 is worked in full on the handout.

2 — Compile XNOR

XNOR is 1 when A and B agree, so it is the OR of the two agreeing rows: $(\neg A \wedge \neg B) \vee (A \wedge B)$. The recipe turns each term into a NOR of its complements, ORs the terms, and inverts:

GENE	IS	COMPUTES
$\neg A$	NOT A	
$\neg B$	NOT B	
P	NOR(A, B)	both low
Q	NOR$(\neg A, \neg B)$	both high
R	NOR(P, Q)	XOR
out	NOT R	XNOR

Six genes. Longest path 4, shortest path 3. Count them: $A \to P \to R \to$ out is three; $A \to \neg A \to Q \to R \to$ out is four. The number most people write down is 3 and 2, from counting the layers in the table above rather than the routes through the network.

A different group's network is not worse for being different. An exhaustive search over networks of two-input NOR gates says the smallest XNOR is **four** gates, with paths of 3 and 2:

$$P = \mathrm{NOR}(A, B), \quad S = \mathrm{NOR}(A, P), \quad T = \mathrm{NOR}(B, P), \quad \mathrm{out} = \mathrm{NOR}(S, T).$$

Two fewer genes to clone and one less protein lifetime of delay, for the same truth table. The table does not choose between them; a compiler does, and that step is "logic synthesis" in the Cello pipeline.

What both have in common, and it is the point of the session: **the paths are not the same length.**

3 — The other circuit

$R_1 = \mathrm{NOT}\ A$, $R_2 = \mathrm{NOR}(B, R_1)$, $\mathrm{out} = \mathrm{NOT}\ R_2$.

t	R_1	R_2	OUT		t	R_1	R_2	OUT
$A=0,\ B=0$					$A=0,\ B=1$			
0	0	0	0		0	0	0	0
1	1	1	**1**		1	1	0	1
2	1	0	**0**		2	1	0	1
3	1	0	**1**		3	1	0	1
4	1	0	1		4	1	0	1
$A=1,\ B=0$					$A=1,\ B=1$			
0	0	0	0		0	0	0	0
1	0	1	**1**		1	0	0	1
2	0	1	**0**		2	0	0	1
3	0	1	0		3	0	0	1
4	0	1	0		4	0	0	1

(a) It settles to

A	B	out
0	0	1
0	1	1
1	0	0
1	1	1

which is $\mathrm{NOT}(A \wedge \neg B)$ — the complement of item 1's classifier.

(b) Two of the four glitch.

- $A=0, B=0$: the output goes 0, 1, 0, 1. It was supposed to change once and changed three times — **dynamic**.
- $A=1, B=0$: the output goes 0, 1, 0. It was supposed to stay at 0 — **static-0**.
- The other two are clean, which is worth noticing: a circuit can be wrong in some input cases and right in others, and testing the one case you happen to try tells you nothing about the rest.

(c) The answer almost every group writes is *the paths are different lengths* — A reaches the output through three genes and B through two — and it is the right instinct and the wrong diagnosis. Padding the short path moves the glitch by one tick and does not remove it; a perfectly balanced circuit built from these gates glitches too.

The real cause: **a repressor gate is ON before its repressor arrives.** At $t=0$ the cell is empty, so every NOT and every NOR in the circuit is expressing, and that false ON walks down the circuit while it fills. The output is a NOT gate, so the first thing it does on being made is turn on.

The fix is not in the wiring. It is in the promoter: make the output **require an activator**, so that an empty cell presents nothing.

4 — The four-gene chain

$$t_{1/2}(4) \;\approx\; \left(4 - \tfrac{1}{3}\right)\tau = \mathbf{3.67}\,\tau, \qquad \text{exactly } 3.672\,\tau.$$

Against the three-gene chain's $2.674\,\tau$, that is **$0.998\,\tau$ later** — one more gene, one more protein lifetime, which is the number to carry out of the session.

Why people underestimate it. The first stage is the fast one: $t_{1/2}(1) = \ln 2 = 0.693\,\tau$, well under a lifetime. Every stage after it costs very nearly a full τ (0.985, 0.996, 0.998, …), so a chain feels cheap when you think about its first gene and is not.

And this is the floor. Hooshangi's measured cascades are far slower: circuit 1 settles by about 120 min and circuit 3, two stages deeper, not until about 600 (p. 3583). Their model has something ours does not — a repressor must clear a **threshold** before the next stage moves at all.

5 — Fail-safe and not balanced, and it still glitches

Take item 1's classifier and pad the short path with an activator, which is the repair most people reach for first:

$$R_1 = \mathrm{NOT}\,B, \quad R_2 = \mathrm{NAND}(A, R_1), \quad \mathrm{Act} = \mathrm{NOT}\,R_2, \quad R_3 = \mathrm{BUF}(\mathrm{Act}).$$

R_3 now needs Act to be **present**, so in an empty cell it is off: the circuit is **fail-safe**. It is still unbalanced — A reaches the output in three genes, B in four. And the output is

$$0,\ 0,\ 1,\ 0,\ 0$$

in **all four** input cases: a static-0 hazard every time. Fail-safety moved the glitch one tick later and did not remove it.

So neither condition is sufficient on its own, and the session has a counterexample for each:

CIRCUIT	OFF WHEN EMPTY?	PATHS EQUAL?	RESULT
the classifier	no	no	glitches 4/4
$Z = \mathrm{NOR}(\neg A, \neg B)$	no	yes	glitches 3/4
this one, padded	yes	no	glitches 4/4
activator required, legs matched	yes	yes	clean

The rule, stated honestly: for an output that does something irreversible, the circuit must be OFF in a cell that has just been made **and** the signals driving that output must arrive at the same depth. The first is a promoter decision and the second is a wiring decision, and you need both.

One caveat worth keeping: this model gives every gate the same unit delay. Real promoters do not, and nothing in the model tells you which way unequal delays push the answer.